Architektury systemów komputerowych: Reference

Data: 2026-06-22
Zakres: 1 - 13

1. Logika cyfrowa

Układ	Formuła / sygnatura	Opis
Sumatory		
Półsumator	`s = x ^ y` `c = x & y`	Dodaje dwa bity. `s` to suma, `c` to bit przeniesienia (carry).
Pełny sumator (FA)	`s = x ^ y ^ ci` `co=x&y|ci&(x^y)`	Jak półsumator, ale przyjmuje też przeniesienie wejściowe `ci`.
Sumator n-bitowy	$n \times$ FA	Kaskadowo wyjście C_i trafia na wejście kolejnego FA. Ostatnie `co` to **Carry Out** całości.
Układy kombinacyjne		
Dekoder	$n \to 2^n$	Wejście koduje liczbę k. Na wyjściu zapalony jest bit k.
Multiplekser	$2^n + n \to 1$	2^n bitów danych + n bitów sterujących S. Na wyjściu zapalony S-ty bit danych.
ALU	$A, B, f \to C$	Dekoder na bitach f wybiera operację np. $A + B$, multipleksowanie wyników bramkami AND/OR.
Układy sekwencyjne (pamięć)		
Przerzutnik S-R	`S` - set, `R` - reset	Przechowuje 1 bit (Q). `S=1` ustawia $Q = 1$, `R=1` zeruje, `S=R=0` trzyma stan. Dwa sprzężone `NOR`-y.
Sterowany poziomem	aktywny gdy `CLK` $= 1$	Wejścia `S` / `R` zAND-owane z zegarem.
Sterowany zboczem	zbocze $1 \to 0$	Dwa przerzutniki poziomowe w kaskadzie. Stan przepisuje się w **momencie** zmiany zegara.

2. Typy danych

Typ	`char`	`short`	`unsigned`	`int`	`long`	`float`	`double`	`void*`
x86-64	1	2	4	4	8	4	8	8

3. Rozszerzanie, obcinanie i przesunięcia

`x << k`	$x \cdot 2^k$ (sgn./uns.)
`u >> k`	$\lfloor u/2^k \rfloor$ - logiczne (zera)
`x >> k`	arytmetyczne (kopia MSB), zaokr. do $-\infty$
$x/2^k$ do 0	`(x + (1<<k)-1) >> k`
Strength reduction	`x*24` $\to$ `(x<<5)-(x<<3)`

4. Operacje i endianness

Negacja U2	`-x = ~x+1` ; `-TMin=TMin` , `-0=0`
Wykr. nadmiaru (`s=x+y`)	`((s^x)&(s^y))>>>(w-1)`
Maska / abs	`m=x>>31; abs=(x^m)-m`
`c ? a : b`	`b ^ (-c & (a ^ b))`

Promocja w C: `unsigned` i `int` w jednym wyrażeniu/porównaniu (`<` `>` `==`) $\to$ `int` promowany do `unsigned` (bity bez zmian, $-1 \to$ UMax).

Schemat	Najniższy adres	`0x0123`
Big Endian	MSB	`[01][23]`
Little Endian	LSB	`[23][01]`

5. Zmiennoprzecinkowe (IEEE 754)

$$V = (-1)^s \times M \times 2^E \qquad \text{Bias} = 2^{k-1} - 1$$

Format	bity	s	exp (k)	frac (n)
`float`	32	1	8	23
`double`	64	1	11	52

Kategorie wartości

Kategoria	`exp`	Własności
Znormalizowane	`!= 0...0` `!= 1...1`	$E = \exp - \text{Bias}$. $M = 1.\text{frac}$. Najgęściej ułożone wokół 0.
Zdenormaliz.	`== 0...0`	$E = 1 - \text{Bias}$. $M = 0.\text{frac}$. Reprezentują ± 0.0 ($\text{frac} = 0$) i liczby bardzo bliskie zera.
Specjalne	`== 1...1`	`frac == 0` $\to \pm\infty$ (nadmiar, dzielenie przez 0). `frac != 0` $\to$ `NaN` (np. $\sqrt{-1}$, $\infty - \infty$).

Zaokrąglanie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).

`... x x G` | `R S S S ...`

`G` - ostatni zachowany, `R` - pierwszy usunięty, `S` - OR reszty

Warunek	Ułamek	Akcja
`R=0`	< 0.5	W dół (odcięcie)
`R=1, S=1`	> 0.5	W górę (+1 do `G`)
`R=1, S=0`	$= 0.5$	**Do parzystej:** w górę gdy `G=1`, w dół gdy `G=0`

Własności matematyczne i rzutowanie

Brak łączności	$(a + b) + c \neq a + (b + c)$ - zaokrąglenia
Brak rozdzielności	$a(b + c) \neq ab + ac$
`int` $\to$ `float`	możliwa utrata precyzji (zaokrągla)
`int` $\to$ `double`	bezstratne ($52 > 32$ bity)
`float/double` $\to$ `int`	obcina do 0; poza zakresem lub `NaN` $\to$ `TMin` (`0x80000000`)

6. Rejestry x86_64

%rax	%eax	%ax %ah %al	%rbx	%ebx	%bx %bh %bl
%rcx	%ecx	%cx %ch %cl	%rdx	%edx	%dx %dh %dl
%rsi	%esi	%si %sil	%rdi	%edi	%di %dil
%rbp	%ebp	%bp %bpl	%rsp	%esp	%sp %spl
%r8	%r8d	%r8w %r8b	%r9	%r9d	%r9w %r9b

Uwaga: Rejestry od %r10 do %r15 używają schematu: `%r10`, `%r10d`, `%r10w`, `%r10b`.

Podział ról rejestrów

`%rax`	Akumulator. Główny rejestr arytmetyczny. Przechowuje **wartość zwracaną**.
`%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`	Kolejno: **od 1. do 6. argumentu funkcji**. Caller-saved.
`%rsp`	Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.
`%rbp`	Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.
`%rbx`, `%r12-%r15`	Ogólnego przeznaczenia. Wszystkie **callee-saved**.
`%rip`	Wskaźnik instrukcji (Program Counter).

Rejestry wektorowe (zmiennoprzecinkowe)

`%xmm0-%xmm15`	128-bitowe. `%xmm0` to wartość zwracana (`float` / `double`). Pierwsze 8 to argumenty. Caller-saved.
`%ymm0-%ymm15`	256-bitowe rozszerzenie XMM. Dzielą z nimi najmłodsze 128 bitów.

7. Tryby adresowania (operandy)

Tryb	Format	Obliczanie adresu
Natychmiastowy (Imm)	`$Imm`	`Imm`
Rejestrowy (Reg)	`%Ra`	`%Ra`
Bezpośredni (Mem)	`Imm`	`M[Imm]`
Pośredni (Mem)	`(%Rb)`	`M[%Rb]`
Z przesunięciem	`D(%Rb)`	`M[%Rb + D]`
Skalowany (Ind/scaled)	`D(%Rb, %Ri, S)`	`M[%Rb+%Ri*S+D]`
Skalowany (baseless)	`(, %Ri, S)`	`M[%Ri * S]`

Legenda: `Imm` / `D` to stała liczbowa, `%Ra` / `%Rb` to rejestr bazowy, `%Ri` to rejestr indeksowy, a `S` to skala (tylko: 1, 2, 4 lub 8).

8. Flagi stanu i sterowanie

`ZF`	Zero. Wynik to 0 (np. argumenty są równe).
`SF`	Sign. Wynik jest ujemny (MSB = 1).
`CF`	Carry. Przepełnienie dla liczb **bez znaku** (unsigned).
`OF`	Overflow. Przepełnienie dla liczb **ze znakiem** (signed).

Sufiksy warunkowe (dla `jX`, `setX`, `cmovX`)

Sufiks	Znaczenie
`e` / `z`	Equal / Zero (równe / wynik to 0)
`ne` / `nz`	Not Equal / Not Zero (nierówne)
`s`	Sign (wynik ujemny, `SF=1`)
Liczby ze znakiem (signed)	
`g` / `ge`	Greater / Greater or Equal
`l` / `le`	Less / Less or Equal
Liczby bez znaku (unsigned)	
`a` / `ae`	Above / Above or Equal (No Carry)
`b` / `be`	Below / Below or Equal (Carry)

Translacja struktur kontrolnych (C → ASM)

- **if-else:** `jX` (skok) lub `cmovX` (liczy obie ścieżki, bez kary za predykcję). `cmovX` odpada przy drogich gałęziach, wyłuskaniach (`*p`) i efektach ubocznych (`x++`).
- `switch`: tablica skoków → czas $O(1)$. Skok pośredni: `jmp *.L4(,%rdi,8)`. Brak `break` ⇒ **fall-through**.
- **Pętla** `for`: zawsze redukowana do `while`: `init; while(cond) { body; update; }`

Wzorce asemblerowe dla pętli:

`do-while`	`while` (Jump-to-mid)	`while` (Guarded)

`loop:`
`  Body`
`  if(T) goto loop`
	`  goto test`
	`loop:`
	`  Body`
	`test:`
	`  if(T) goto loop`
		`  if(!T) goto done`
		`loop:`
		`  Body`
		`  if(T) goto loop`
		`done:`

9. Procedury i stos

Stos rośnie **w dół** (niższe adresy); `%rsp` wskazuje **wierzchołek** (najniższy zajęty adres). `push` zmniejsza `%rsp` o 8, `pop` zwiększa o 8.

- `call dest`: odkłada adres powrotu na stos i skacze do `dest`.
- `ret`: zdejmuje adres powrotu ze stosu i skacze pod niego.

10. Konwencja Wywoływań (System V ABI)

`%rdi` → `%rsi` → `%rdx` → `%rcx` → `%r8` → `%r9`

Argumenty 7+: Na stosie od końca. Wynik w `%rax`.

Rejestry caller-saved vs callee-saved

Caller-saved
(Wołający musi zapisać)
Mogą zostać nadpisane w funkcji. By je zachować, caller kładzie je na stos przed `call`.

`%rax`, `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` - `%r11`

Callee-saved
(Wołany musi przywrócić)
Muszą zachować stan. Callee musi zapisać je na stos i odtworzyć przed `ret`.

`%rbx`, `%rbp`, `%r12` - `%r15`

Ramka stosu

Każde wywołanie funkcji ma własną ramkę (stąd **rekurencja** działa naturalnie).

Ramka potrzebna, gdy: brak rejestrów na zmienne (spilling), lokalna tablica/struktura, lub pobrano adres zmiennej (`&`).

Prolog

```
pushq %rbp          ;
zapisz stary base ptr
movq  %rsp, %rbp    ;
nowy base ptr = rsp
subq  $32, %rsp     ;
alokacja 32 bajtów
```

Epilog

```
addq  $32, %rsp     ;
zwolnij alokacj
popq  %rbp          ;
przywró  base ptr
ret                 ;
skok powrotny
```

`leave`: zastępuje `movq %rbp, %rsp` + `popq %rbp` jedną instrukcją.

11. Architektura CPU

Fazy przetwarzania instrukcji

Fetch → Decode → Execute → Memory → Write

Pipelining i hazardy

Nakładanie faz zwiększa throughput, ale rodzi konflikty (hazardy):

Danych	Odczyt po zapisie – wynik jeszcze nie jest w rejestrze, a kolejna instrukcja już go potrzebuje. **Rozwiązanie:** forwarding/bypassing (wynik z ALU prosto na wejście) lub stall/bubble.
Sterowania	Skok wymusza odgadnięcie następnego adresu. **Rozwiązanie:** branch prediction. Pomyłka → czyszczenie potoku (misprediction penalty).

Out-of-Order

Nowoczesne procesory nie wykonują instrukcji sekwencyjnie:

- **Superskalarność:** wiele instrukcji na cykl (wiele jednostek wykonawczych).
- **Register renaming:** mapowanie rejestrów logicznych (`%rax`) na wiele fizycznych – eliminuje fałszywe zależności.
- **Reorder buffer:** instrukcje liczone asynchronicznie, ale zatwierdzane w kolejności programu (spójność).

Miary wydajności

Latency bound	Limit wynikający z łańcucha zależności danych. Zależy od opóźnienia jednostki. Np. wynik z poprzedniej iteracji jest potrzebny w obecnej.
Throughput bound	Limit wynikający z przepustowości sprzętu.

Fazy przetwarzania

Dispatch	Dekodowanie i alokacja w stacji rezerwacyjnej. CPU może zlecić ograniczoną liczbę instrukcji na cykl.
Execute	Wykonywanie właściwe. Zależne instrukcje mogą rozpocząć `e` w cyklu udostępnienia wyniku (`w`) przez instrukcję poprzedzającą.
Write-back	Udostępnianie wyniku na magistrali. W tym samym cyklu zależne instrukcje zaczynają `e`.
Retire	Zatwierdzenie stanu architektury. Musi zachodzić ściśle in-order.

12. Tablice i struktury

Reprezentacja tablic w pamięci

Typ	Deklaracja	Adres
Jednowymiarowa	`T A[L]`	$A[i] = x_A + i \times \text{sizeof}(T)$
Wielowymiarowa	`T A[R][C]`	$A[i][j] = x_A + (i \times C + j) \times \text{sizeof}(T)$
Wielopoziomowa	`T *A[L]`	$M[x_A + i \times 8] + j \times \text{sizeof}(T)$ (dwa odczyty z pamięci)

Wyrównanie danych i padding

- **Zasada ogólna:** obiekt rozmiaru K leży pod adresem podzielnym przez K.
- **Struktury:** każde pole wyrównane do własnego K; rozmiar całości podzielny przez największe wyrównanie (padding na końcu).
- **Optymalizacja:** pola od największego do najmniejszego → minimalny padding.

13. Układ pamięci

Mapa pamięci procesu

Stack ↓	Rośnie w dół. Zmienne lokalne, adresy powrotu (limit 8MB).
Shared libs	Współdzielone biblioteki (np. `libc.so`).
Sterta ↑	Rośnie w górę. Dynamiczna alokacja (`malloc`, `new`).
Data	Zmienne globalne i statyczne (zainicjowane i niezainicjowane).
Text	Read-only. Binarny kod wykonywalny programu.

Góra tabeli = wysokie adresy, dół = niskie adresy.

Zagrożenia i mechanizmy obronne

Buffer overflow	Brak sprawdzania granic tablic. Nadpisuje stos (stary `%rbp`, **adres powrotu**) → przejęcie sterowania.
ROP (Gadżety)	Return-Oriented Programming. Sklejanie legalnych skrawków kodu zakończonych `ret` – omija zakaz wykonywania (NX).
Stack canaries	Losowa wartość tuż przed adresem powrotu. Przed `ret` weryfikacja (`xor` + `je`); uszkodzona → `abort()`.
NX / ASLR	**NX:** zakaz wykonywania kodu ze stosu i sterty. **ASLR:** losowanie bazowych adresów obszarów przy każdym starcie.

Unie

Wszystkie pola współdzielą **ten sam adres** (offset 0); rozmiar = **największe pole.** Służą do manipulacji bitowych z ominięciem systemu typów (np. bity `float` jako `int`).

14. Optymalizacje i ich ograniczenia

Kompilator nie zoptymalizuje kodu, jeśli mogłoby to zmienić zachowanie programu (choćby dla specyficznych argumentów).

Optymalizacyjne blokady

Aliasing pamięci	`*p` i `*q` mogą wskazywać na to samo → wymuszony odczyt/zapis do RAM zamiast rejestru. Pomaga słowo kluczowe `restrict`.
Wywołania funkcji	Funkcja w pętli może mieć ukryte efekty uboczne – nie zostanie wyrzucona poza pętlę. **Rozwiązanie:** inlining lub makra.
Arytmetyka `float`	Brak łączności: $(a+b)+c \neq a+(b+c)$. Kompilator **nigdy** nie zmieni kolejności działań (troska o precyzję).

Podstawowe techniki optymalizacji

Code motion	Wyrzucenie niezmienników (np. `x * y`) poza pętlę.
Strength reduction	Kosztowne → tańsze (np. `x * 64` → `x << 6`, iteracja wskaźnikiem zamiast mnożenia indeksu).
Share subexpr.	Jednokrotne obliczanie powtarzających się podwyrażeń.
Loop unrolling	Rozwinięcie pętli (np. krok co 2). Mniejszy narzut pętli, więcej równoległości dla CPU.

15. Linkowanie i konsolidacja

Fazy budowania programu

`cpp` (Preprocesor) → `cc1` (Kompilator) → `as` (Asembler) → `ld` (Linker)

Formaty plików obiektowych (ELF)

- **Relokowalne (`.o`):** kod i dane gotowe do połączenia z innymi `.o`.
- **Wykonywalne (`a.out`):** sklejone, gotowe do załadowania i uruchomienia.
- **Współdzielone (`.so`):** linkowane dynamicznie przy ładowaniu lub w trakcie działania.

`.text`	Skompilowany kod maszynowy.
`.rodata`	Dane tylko do odczytu (np. `"Witaj"` w `printf`, tablice `switch`).
`.data`	Zainicjowane zmienne globalne i statyczne (np. `int x = 5;`).
`.bss`	Niezainicjowane globalne/statyczne – nie zajmują miejsca w pliku.
`.symtab`	Tablica symboli (funkcje i zmienne globalne).
`.rel.text` / `.data`	Wpisy relokacji: gdzie i jak linker ma wstawić ostateczne adresy.

Symbol resolution

Linker rozróżnia 3 typy symboli:

- **Globalne:** zdefiniowane tu, używane tam,
- **Zewnętrzne:** `extern` – używane tu, zdefiniowane gdzie indziej,
- **Lokalne:** z C-owym słowem `static`.

Uwaga: Zmienne lokalne na stosie **NIE** są w ogóle widoczne dla linkera

Silne (Strong): Funkcje i zainicjowane zmienne globalne (np. `int foo = 5;`).
Słabe (Weak): Niezainicjowane zmienne globalne (np. `int foo;`).

Relokacja

Linker skleja sekcje `.text` / `.data` z plików `.o` w jeden blok, nadaje finalne adresy (run-time) i poprawia wszystkie odniesienia w kodzie wg wpisów z `.rel`.

Biblioteki

Statyczne (`.a`)
Zbiór plików `.o`. Linker kopiuje **tylko używane moduły**.
Wady: każda aplikacja ma własną kopię w RAM, zmiana wymusza relink.
Flagi `gcc`: `.a` podaje się na końcu komendy.

Dynamiczne (`.so`)
Kod ładowany do pamięci raz; adresy dostarczane w trakcie działania (przez GOT).
Zalety: znacznie mniejsze binarki, łatwe aktualizacje.

Identyfikacja relokacji w kodzie

Linker musi załatać adresy (wstawić relokacje) w każdym miejscu, gdzie kod:

- wywołuje funkcję zdefiniowaną w innym pliku/module (np. `printf()`),
- odwołuje się do zmiennej globalnej (nawet zadeklarowanej w tym samym pliku),
- odwołuje się do statycznej zmiennej lokalnej (`static int counter;`),
- używa literału znakowego (np. `"%d "`), który trafia do sekcji `.rodata`.

16. Dynamiczna alokacja

Rodzaje fragmentacji

Wewnętrzna	Przydzielony blok jest **większy** niż zażądane dane.
Zewnętrzna	Wolnego miejsca jest dość w sumie, ale jest **rozrzucone**.

Budowa bloku

Header	Dane	Footer
Rozmiar + a	+ Padding	(Boundary tag)

Polityki przydziału i łączenia

First fit	Pierwszy pasujący od początku. Szybki, fragmentuje początek sterty.
Next fit	Od miejsca ostatniego przydziału. Szybszy, gorsza fragmentacja.
Best fit	Blok najbliższy żądanemu rozmiarowi. Najlepsza pamięć, najwolniejszy.
Coalescing	Łączenie sąsiednich wolnych bloków przy `free()`. Footery $\rightarrow$ sprawdzenie poprzednika w $O(1)$.

17. Hierarchia pamięci i lokalność

SRAM vs DRAM

SRAM
Bardzo szybka, droga. Trzyma stan dopóki jest zasilanie (bez odświeżania).

DRAM
Wolniejsza, tania, pojemna. Wymaga ciągłego odświeżania.

Lokalność

- **Czasowa:** użyte dane zaraz będą użyte ponownie (np. licznik pętli).
- **Przestrzenna:** po adresie x zaraz przyjdą $x+1, x+2$ (np. iteracja po tablicy).

Rodzaje chybień

Cold (compulsory)	Blok pobierany z RAM **po raz pierwszy**.
Capacity	Working set programu **większy** niż pojemność cache.
Conflict	Różne bloki mapują się na **ten sam set** i nawzajem się wypierają, mimo wolnego miejsca gdzie indziej.

18. Pamięć podręczna

Parametry i pojemność

$S = 2^s$ (liczba zbiorów), E (liczba linii w zbiorze), $B = 2^b$ (bajtów na blok danych)
Całkowity rozmiar pamięci podręcznej (bez tagów): $C = S \times E \times B$

Podział adresu sprzętowego

Tag	Indeks	Offset
t bitów	s bitów	b bitów

- **Tag:** Identyfikator bloku. Sprawdzany w $O(1)$ dla linii w danym zbiorze.
- **Indeks:** Wskazuje sprzętowo na zbiór.
- **Offset:** Konkretny bajt wewnątrz bloku danych.
- **Valid bit:** Czy linia ma poprawne dane (1), czy śmieci (0).

Asocjatywność

Direct-mapped	$E = 1$. Adres pasuje do **dokładnie jednej** linii. Szybkie, ale adresy o tym samym indeksie się wypierają.
E-way assoc.	$E > 1$. Blok ma zbiór, ale w nim **dowolną** linię. Wymaga strategii wymiany (np. LRU).
Fully assoc.	$S = 1$. Linia może trafić **gdziekolwiek**. Bardzo drogie sprzętowo.

Polityki zapisu

Write-hit
- **Write-through:** równoczesny zapis do cache i RAM.
- **Write-back:** zapis tylko do cache + **dirty bit**. RAM aktualizowany przy wyrzuceniu linii.

Write-miss
- **Write-allocate:** wciąga blok z RAM do cache, potem nadpisuje. Para z write-back.
- **No-write-allocate:** zapis prosto do RAM, z pominięciem cache. Para z write-through.

19. Pamięć wirtualna

Translacja adresów (VA $\rightarrow$ PA)

Virtual Address	Physical Address
`VPN` | `VPO`	`PPN` | `PPO`

- **VPO = PPO:** Offset **nigdy się nie zmienia** przy translacji.
- Sprzętowy układ MMU zamienia `VPN` na `PPN` używając **tablicy stron**.

Tablice stron i TLB

- **Page fault:** dostęp do strony spoza RAM $\rightarrow$ wyjątek; OS wstrzymuje proces i wczytuje stronę z dysku.
- **TLB:** mały sprzętowy cache translacji (`VPN` $\rightarrow$ `PPN`) wewnątrz CPU - eliminuje odczyt tablicy stron z pamięci przy każdym żądaniu.
  - **Struktura TLB:** Jeżeli TLB jest pamięcią **w pełni asocjacyjną**, to `VPN` w całości stanowi tag TLB. W przeciwnym wypadku, `VPN` dzieli się na tag oraz dodatkowo na indeks, który określa konkretny zbiór wewnątrz TLB.

VIPT (Virtually Indexed, Physically Tagged)

Gdy L1 jest małe, indeks (`CI`) mieści się w `VPO`, a `VPO = PPO`:

`VPO` $\rightarrow$ indeks L1 $\rightarrow$ równolegle: `VPN` $\rightarrow$ TLB $\rightarrow$ tag `CT` weryfikuje wiersz

Lookup zbioru w L1 startuje **równolegle** z translacją w TLB, co skraca czas dostępu.

20. Katalog instrukcji (AT&T)

Opcode	Efekt	Opis
`mov S, D`	$D \leftarrow S$	Kopiuje wartość z S do D.
`lea S, D`	$D \leftarrow \text{addr}(S)$	Oblicza adres S (bez czytania pamięci).
`add S, D`	$D \leftarrow D + S$	Dodawanie (ustawia flagi).
`sub S, D`	$D \leftarrow D - S$	Odejmowanie (ustawia flagi).
`imul S, D`	$D \leftarrow D * S$	Mnożenie liczb ze znakiem.
`sal / shl k, D`	$D \ll= k$	Przesunięcie w lewo (mnożenie przez 2^k).
`sar k, D`	$D \gg= k$	Arytmetyczne w prawo (dzielenie). **Kopiuje znak**.
`shr k, D`	$D \gg= k$	Logiczne w prawo. Dopełnia zerami.
`and / or S, D`	$D \leftarrow D \ \& \ / \ \mid S$	Operacje bitowe (ustawiają flagi).
`xor S, D`	$D \leftarrow D \hat{} S$	Często jako `xor %rax, %rax` do zerowania.

Porównania i sterowanie

`cmp S1, S2`	$\text{S2} - \text{S1}$	Ustawia flagi jak odejmowanie, nie zapisuje wyniku.
`test S1, S2`	$\text{S2} \ \& \ \text{S1}$	Ustawia flagi jak **AND**.
`jX dest`	$\text{if}(X) \ \%\text{rip} \leftarrow \text{dest}$	Skok warunkowy (X = warunek).
`setX D`	$\text{if}(X) D \leftarrow 1$	Ustawia najmłodszy bajt na 0 lub 1 wg flag.
`cmovX S, D`	$\text{if}(X) D \leftarrow S$	Warunkowe kopiowanie (zamiast skoku).

Stos i zarządzanie procedurami

`push S`	$\%\text{rsp} -= 8$ $M[\%\text{rsp}] \leftarrow S$	Odkłada na stos (zmniejsza `%rsp`).
`pop D`	$D \leftarrow M[\%\text{rsp}]$ $\%\text{rsp} += 8$	Zdejmuje ze stosu do D (zwiększa `%rsp`).
`call dest`	$\text{push} \ \%\text{rip}$ $\%\text{rip} \leftarrow \text{dest}$	Skok do funkcji (zapisuje adres powrotu na stosie).
`ret`	$\text{pop} \ \%\text{rip}$	Zdejmuje adres powrotu ze stosu i skacze pod niego.
`leave`	$\%\text{rsp} \leftarrow \%\text{rbp}$ $\text{pop} \ \%\text{rbp}$	Sprząta ramkę stosu (odwrotność prologu).

Operacje wektorowe

Rozszerzenie AVX. Przedrostek `v` (nie niszczy źródeł).

`vmovaps / ups S, D`	$D \leftarrow S$	Kopiuje wektor. `a` (aligned – szybkie, adres podzielny przez wyrównanie), `u` (unaligned).
`vaddps / pd S1, S2, D`	$D \leftarrow \text{S2} + \text{S1}$	Dodawanie wielokrotne (packed). `ps` (single-float), `pd` (double).
`vmulps / pd S1, S2, D`	$D \leftarrow \text{S2} * \text{S1}$	Mnożenie wektorowe.
`vfmadd231ps S1, S2, D`	$D \leftarrow \text{S2} * \text{S1} + D$	**Fused Multiply-Add**. Mnoży i dodaje do `D` w jednym kroku.

Operacje zmiennoprzecinkowe

`movsd / movss S, D`	$D \leftarrow S$	Kopiuje skalar (`double` / `float`) między rejestrami XMM lub pamięcią.
`movapd / movaps S, D`	$D \leftarrow S$	Kopiuje wyrównany wektor zmiennoprzecinkowy.
`addsd / subsd S, D`	$D \leftarrow D \ \text{op} \ S$	Skalarne dodawanie / odejmowanie podwójnej precyzji.
`xorpd / xorps S, D`	$D \leftarrow D \hat{} S$	Bitowy XOR na XMM. Jako `xorpd %xmm0, %xmm0` zeruje rejestr.
`ucomisd S1, S2`	$\text{S2} - \text{S1}$	Porównuje `double` i ustawia flagi `ZF, PF, CF` w `%rflags`.

Uwaga o sufiksach: Instrukcje przyjmują sufiks określający rozmiar danych: `b` (8-bit), `w` (16-bit), `l` (32-bit), `q` (64-bit).
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).